

Sistemi Operativi (Modulo II)

Capitolo 8 — Sicurezza

Luca Argentino

A.A. 2025–2026

Indice

8	Sicurezza	3
8.1	Introduzione	3
8.1.1	Security e Safety	3
8.1.2	Principi pratici di sicurezza	3
8.1.3	Protezione, Sicurezza e Trust	4
8.1.4	Obiettivi della sicurezza: la triade CIA	5
8.1.5	Politiche e meccanismi	6
8.2	Crittografia	7
8.2.1	Definizioni e schema generale	7
8.2.2	Funzioni a senso unico (OWF)	7
8.2.3	Crittografia a chiave privata (o segreta)	8
8.2.4	Crittografia a chiave pubblica (asimmetrica)	9
8.3	Attacchi	10
8.3.1	Tipologie di attacco	10
8.3.2	Obiettivi degli attaccanti	10
8.3.3	Elementi coinvolti negli attacchi	10
8.3.4	Percorsi di attacco	11
8.3.5	Esempio di attacco banale	11
8.4	Vulnerabilità del software	13
8.4.1	Buffer Overflow	13
8.4.2	Errori Tempo Controllo / Tempo Utilizzo (TOCTOU)	15
8.5	Codice Maligno	16
8.5.1	Trojan Horse (Cavalli di Troia)	16
8.5.2	Bombe Logiche	16
8.5.3	Backdoor e Trapdoor	17
8.5.4	Virus e Worm	17
8.5.5	Internet Worm di Morris (1988)	20
8.6	Autenticazione	22
8.6.1	Autenticazione tramite qualcosa che l'utente conosce: password	22
8.6.2	Attacchi alle password	24
8.6.3	Password One-Shot	25
8.6.4	Autenticazione con oggetti fisici e biometrica	25
8.6.5	PAM – Pluggable Authentication Modules	26
8.7	Controllo degli Accessi: Autorizzazione	27
8.7.1	Protezione del nucleo e autorizzazione	27
8.7.2	Principi fondamentali	27
8.7.3	Definizioni: soggetti, oggetti, diritti, dominio	28

8.7.4	Matrice di Accesso	28
8.7.5	Revoca dei diritti di accesso	29
8.7.6	Modello tradizionale UNIX (modello ACL limitato)	31
8.7.7	POSIX ACL	34
8.7.8	POSIX Capability	38
8.7.9	DAC e MAC	39
8.7.10	MAC – Modello Bell-LaPadula	40

Capitolo 8

Sicurezza

8.1 Introduzione

8.1.1 Security e Safety

In italiano entrambi i termini si traducono come *sicurezza*, ma in inglese denotano concetti distinti:

Definizione – Security e Safety

- **Security**: resilienza ad attività *volontarie* e intenzionali (dolo).
Un sistema è *secure* se resiste ad attacchi deliberati.
- **Safety**: resilienza a eventi *accidentali* e non intenzionali (colpa).
Un sistema è *safe* se non causa danni a causa di guasti o errori involontari.

8.1.2 Principi pratici di sicurezza

Le seguenti considerazioni fondamentali sono rielaborate dai principi enunciati in Cheswick, Bellovin, Rubin – *Firewalls and Internet Security* (Addison-Wesley, 2003):

- **Non esiste un sistema completamente sicuro.** Non è possibile garantire la sicurezza assoluta; è però possibile rendere un attacco sufficientemente costoso da scoraggiarlo.
- **La sicurezza non è uno stato, ma un processo continuo.** Richiede tre componenti:
 - Hardware;
 - Software;
 - *Humanware* (lavoro umano competente, non automatizzabile).
- **KISS – Keep It Simple and Stupid!** Ogni programma ha una probabilità di avere un bug pari ad uno.

Le fragilità di un sistema crescono con la sua complessità in modo più che proporzionale. Sistemi complessi hanno più superfici di attacco.

- **Sicurezza a livelli/strati.** Gli attaccanti non affrontano la sicurezza frontalmente: la aggirano. Occorre implementare difese in profondità (*defense in depth*).
- **Principio del privilegio minimo.** Non si devono assegnare a programmi o persone autorizzazioni maggiori di quelle strettamente necessarie.
- **La catena è forte quanto il suo anello più debole.**

Le tipologie di phishing vedono gli utenti spesso come l'anello più vulnerabile.

Attenzione – Security by Obscurity

La "security by obscurity" è una cattiva idea, non funziona. Cercare di ottenere protezione mantenendo segreti gli algoritmi di crittografia è un approccio debole, poiché non c'è nulla di più insicuro di una falsa sensazione di sicurezza. In un sistema robusto, la sicurezza viene ottenuta mantenendo segrete le chiavi, non gli algoritmi.

L'algoritmo, al contrario, può essere pubblico. Un codice sorgente visibile a tutti permette a un maggior numero di persone di analizzarne il comportamento e individuare eventuali problemi. La trasparenza, quindi, non rende il sistema più vulnerabile, ma aiuta a migliorarne la robustezza.

Nota – Sicurezza come questione economica

Il costo delle contromisure di sicurezza deve essere commisurato al valore delle informazioni protette. Investire più del valore del bene da proteggere non è razionale; d'altro canto, investire troppo poco espone a rischi sproporzionati.

8.1.3 Protezione, Sicurezza e Trust

Definizione – Sicurezza, Trust e Protezione

- **Sicurezza:** il problema generale, che coinvolge non solo il sistema informatico ma anche aspetti amministrativi, legali, politici e finanziari. È un concetto *assoluto*: il sistema è sicuro oppure non lo è.
- **Trust:** la misura della *fiducia* sulla sicurezza di un sistema informatico. È un concetto *relativo*: esistono diversi gradi di fiducia.
- **Protezione:** l'insieme dei meccanismi implementati per proteggere concretamente il sistema informativo.

8.1.4 Obiettivi della sicurezza: la triade CIA

La sicurezza di un sistema informatico si articola attorno a tre obiettivi fondamentali, spesso rappresentati come una triade:

Definizione – Triade CIA

- **Confidentiality** (Riservatezza): come mantenere la segretezza dei dati, garantendo che solo i soggetti autorizzati possano accedervi.
- **Integrity** (Integrità): come evitare che i dati vengano alterati in modo non autorizzato.
- **Availability** (Disponibilità): come garantire che il sistema continuerà ad operare e a fornire i propri servizi.

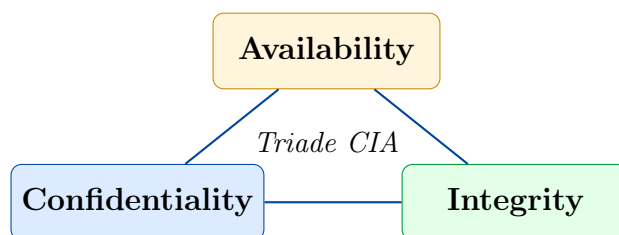


Figura 8.1: I tre obiettivi fondamentali della sicurezza informatica. Alcuni di essi possono essere in contrasto reciproco (ad esempio, garantire la massima disponibilità può ridurre la riservatezza).

Nota – Violazioni alla sicurezza

Un sistema è **sicuro** se tutte le sue risorse sono accedute nei modi previsti e autorizzati. Le violazioni si classificano in:

- **Disclosure** (furto di informazione): attacco alla *confidentiality*.
- **Alteration** (modifica non autorizzata dei dati): attacco all'*integrity*.
- **Denial of Service** (negazione del servizio e della continuità): attacco all'*availability*.

Le effrazioni alla sicurezza informatica sono reati riconosciuti anche dal codice penale italiano.

8.1.5 Politiche e meccanismi

Un concetto fondamentale di ingegneria del software è la **separazione della politica dai meccanismi**:

Definizione – Security Policy e Meccanismi

- **Security Policy** (politica di sicurezza): descrive formalmente o informalmente i *requisiti* di sicurezza del sistema. Risponde alla domanda “*cosa?*”.
- **Meccanismi**: implementano concretamente la security policy. Rispondono alla domanda “*come?*”.

Questo disaccoppiamento permette di:

- Cambiare la politica senza modificare i meccanismi (es. aggiornare le regole di accesso);
- Cambiare i meccanismi senza modificare la politica (es. sostituire l'algoritmo di cifratura).

La scelta di una politica di sicurezza dipende dal tipo di attacchi e attaccanti attesi, dal valore delle informazioni nel sistema e dai costi operativi della politica stessa.

8.2 Crittografia

8.2.1 Definizioni e schema generale

Definizione – Crittografia

La crittografia è l'insieme dei meccanismi utilizzati per trasformare messaggi in chiaro (*plaintext*) in messaggi cifrati (*ciphertext*), leggibili solo a chi possiede le opportune autorizzazioni. L'autorizzazione assume la forma di una **chiave** (K) necessaria per la conversione.

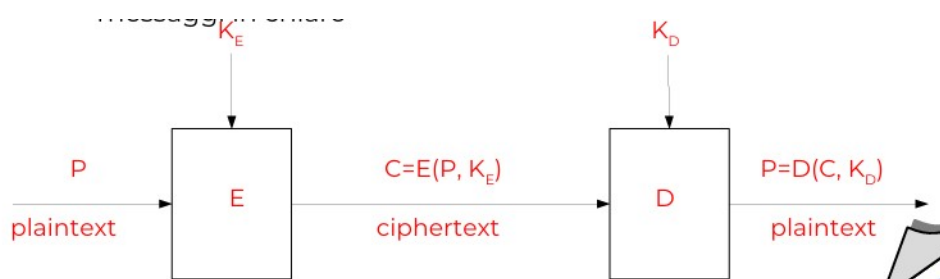


Figura 8.2: Schema generale della crittografia. Il plaintext P viene cifrato dalla funzione E con la chiave K_E , producendo il ciphertext $C = E(P, K_E)$. La funzione D decifra con la chiave K_D , ottenendo $P = D(C, K_D)$.

8.2.2 Funzioni a senso unico (OWF)

A seguire utilizzeremo questo tipo di funzioni descritte in questa sezione:

Definizione – One-Way Function (OWF)

Una **funzione a senso unico** $y = f(x)$ ha le seguenti proprietà:

- Dato x , calcolare $f(x)$ è *relativamente semplice*.
- Dato $f(x)$, calcolare x è *computazionalmente difficile* o praticamente impossibile.

Queste funzioni “mescolano” i bit in modo molto complesso tramite operazioni di bit swapping, inversioni e multiple iterazioni.

Seguire l'algoritmo è semplice; invertirlo è computazionalmente proibitivo.

8.2.3 Crittografia a chiave privata (o segreta)

Definizione – Crittografia simmetrica

Nella **crittografia a chiave privata** (o simmetrica), la stessa chiave viene usata sia per cifrare che per decifrare i messaggi. La chiave deve essere mantenuta segreta da entrambe le parti comunicanti.

- **Vantaggi:** gli algoritmi sono molto veloci e implementabili in hardware.
- **Svantaggi:** la distribuzione sicura della chiave è un problema critico; può richiedere uno scambio fisico.

UTILIZZO: useremo questo meccanismo per la condivisione di messaggi molto grandi

DES – Data Encryption Standard

Il **ESEMPIO: DES** è un algoritmo a chiave segreta a 56 bit. Il suo schema di funzionamento, detto *Feistel cipher*, è il seguente:

1. Data una $OWF(k, x)$ con k chiave a 56 bit e x blocco a 64 bit.
2. Un messaggio m di 64 bit viene diviso in L_0 (32 bit più significativi) e R_0 (32 bit meno significativi).
3. Si applica iterativamente per 16 round:

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus OWF(k, R_{i-1})$$

4. Il ciphertext è dato da L_{16} e R_{16} .

La chiave a soli 56 bit rende il DES oggi vulnerabile ad attacchi a forza bruta; il **3DES** e l'**AES** sono i successori moderni.

8.2.4 Crittografia a chiave pubblica (asimmetrica)

Definizione – Crittografia asimmetrica

Nella **crittografia a chiave pubblica** esistono *due chiavi distinte*:

- **Chiave pubblica** K_E : usata per cifrare il messaggio; può (e deve) essere pubblicata.
- **Chiave privata** K_D : usata per decifrare il messaggio; deve essere mantenuta assolutamente segreta.

Le due chiavi sono collegate matematicamente, ma **dalla chiave pubblica deve essere praticamente impossibile risalire a quella privata.**

ESEMPIO: RSA – Schema di funzionamento

RSA si basa sulla difficoltà computazionale della fattorizzazione di interi molto grandi:

1. Si scelgono due numeri primi molto grandi p e q (almeno 100 cifre ciascuno).
2. Si calcola $n = pq$.
3. Si sceglie d tale che $\gcd(d, (p-1)(q-1)) = 1$.
4. Sia e l'inverso moltiplicativo di d modulo $(p-1)(q-1)$, ovvero $de \equiv 1 \pmod{(p-1)(q-1)}$.
5. **Cifratura**: $C = m^e \bmod n$.
6. **Decifratura**: $m = C^d \bmod n$.

La chiave pubblica è la coppia (e, n) ;

Nota – Sicurezza basata su chiavi vs security by obscurity

La sicurezza moderna si basa sul *mantenere segrete le chiavi*, non gli algoritmi. Gli algoritmi pubblici possono essere analizzati dalla comunità scientifica, aumentando la fiducia nella loro robustezza. La *security by obscurity* – nascondere gli algoritmi – non è una garanzia affidabile.

8.3 Attacchi

8.3.1 Tipologie di attacco

Gli attacchi possono essere classificati secondo diverse dimensioni:

Per motivazione:

- Crimine di opportunità vs attacco con obiettivo specifico.
- Motivazione finanziaria vs motivazione politica.
- Divertimento (nessun movente specifico).
- Attacchi attivi (il sistema attaccato viene modificato) vs attacchi passivi (solo osservazione, senza perturbare sistema attaccato).

Per metodologia:

- **Attacchi interni:** effettuati da software o utenti “all’interno” del sistema protetto; possono seguire un attacco esterno che ha già compromesso parte del sistema.
- **Attacchi esterni:** condotti tramite interfacce di comunicazione con l’esterno (rete, ma anche supporti fisici come CD, chiavette USB, etc.).

8.3.2 Obiettivi degli attaccanti

Definizione – Obiettivi tecnici di un attaccante

Dal punto di vista tecnico, l’obiettivo finale di un attaccante è:

1. Acquisire una qualche forma di controllo sulla macchina.
2. Idealmente, acquisire il **controllo totale** (privilegi di *root/administrator*).

Nei sistemi multi-utente esiste una distinzione critica:

- **Utenti normali:** hanno accesso solo a un sottoinsieme di risorse personali.
- **Superutente (root/administrator):** ha accesso illimitato all’intero sistema.

8.3.3 Elementi coinvolti negli attacchi

- **Hardware:** fornisce i meccanismi base per la protezione (mode bit, protezione memoria, processore e dispositivi). L’ambiente fisico della macchina è anch’esso un elemento da proteggere.
- **Sistema operativo (nucleo):** fornisce i due meccanismi fondamentali di *autenticazione* (sapere chi fa cosa) e *autorizzazione* (chi può fare cosa?).
- **Librerie e tool di sistema:** spesso eseguiti in modalità superutente e contenenti grandi quantità di codice; costituiscono un’ampia superficie di attacco.
- **Applicazioni:** possono fornire una prima “testa di ponte” per attaccare il sistema.

- **Utenti:** l'anello più debole dell'intera catena di sicurezza.

8.3.4 Percorsi di attacco

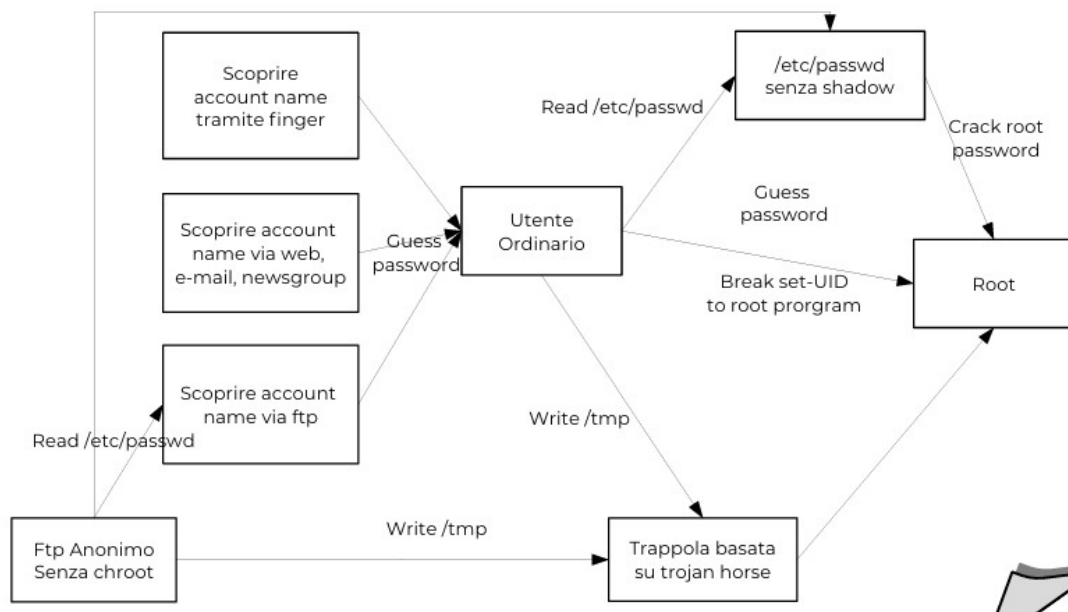


Figura 8.3: Grafo dei possibili percorsi di attacco verso i privilegi di *root*. Partendo da tecniche di raccolta informazioni (finger, web, ftp), l'attaccante può arrivare a compromettere il sistema passando per account ordinari, file */etc/passwd* senza shadow, o trojan horse in */tmp*.

I percorsi principali mostrati nel grafo sono:

- Scoprire nomi di account via **finger**, web, e-mail o FTP anonimo.
- Tentare di indovinare la password (*guess password*).
- Se */etc/passwd* è senza shadow, leggere le password codificate e applicare il cracking.
- Sfruttare programmi con bit SETUID impostato su *root* per scalare i privilegi.
- Installare un trojan horse in */tmp* per intrappolare il superutente.

8.3.5 Esempio di attacco banale

Il seguente esempio illustra un attacco semplice ma realistico:

1. **Individuare il bersaglio** tramite web o e-mail. Verificarne l'esistenza con ping.
2. **Raccogliere informazioni** tramite **finger**: si scopre che il servizio è attivo e che *root* non è connesso.
3. **Enumerare gli account** usando **finger** con un dizionario di nomi comuni.
4. **Tentare il guess della password** su un account utente ordinario.

5. **Scalare i privilegi a root:** ispezionare il `$PATH` del superutente e installare trojan horse (es. una versione maligna di `ls` o `vi`) in `/tmp` per catturare le credenziali di root quando esegue comandi nella directory corrente.

Attenzione – PATH del superutente

Se il `$PATH` di root contiene “.” (directory corrente), un attaccante che scriva un eseguibile maligno con lo stesso nome di un comando di sistema (es. `ls`) nella directory corrente potrà farlo eseguire da root. Questa configurazione è estremamente pericolosa.

8.4 Vulnerabilità del software

Le vulnerabilità del software si dividono principalmente in:

- **Errori di programmazione:** buffer overflow, errori tempo controllo/tempo utilizzo.
- **Codice maligno:** trojan horse, bombe logiche, backdoor, virus, worm.

8.4.1 Buffer Overflow

Definizione – Buffer Overflow

Un **buffer overflow** si verifica quando si fornisce a un programma un input di dimensioni superiori a quelle del buffer allocato per contenerlo. Nel migliore dei casi, il programma va in crash (*segmentation fault*); nel peggiore, l'attaccante prende il controllo della macchina.

Più del 50% degli incidenti riportati al CERT sono storicamente dovuti a buffer overflow.

Meccanismo di exploit

Per comprendere l'exploit è necessario conoscere l'organizzazione della memoria:

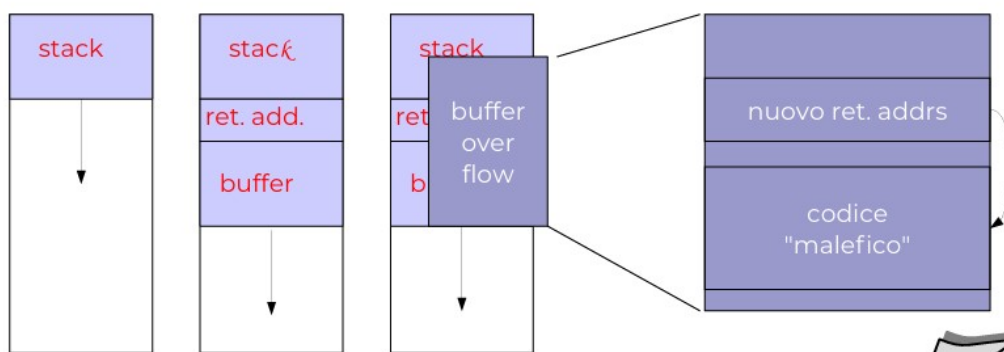


Figura 8.4: Rappresentazione del buffer overflow: l'attaccante riempie il buffer con codice maligno e sovrascrive l'indirizzo di ritorno (*return address*) con un puntatore a tale codice. Al ritorno dalla funzione, la CPU esegue il codice dell'attaccante.

Ad ogni chiamata di funzione, lo stack contiene: indirizzo di ritorno, argomenti della funzione, dati locali. Un programma che non controlla la dimensione dei propri buffer può essere sfruttato per sovrascrivere questi dati.

Funzioni C pericolose

Listing 8.1: Esempio di codice vulnerabile a buffer overflow

```
1 int main() {  
2     char buffer[10];  
3     printf("Inserisci il CAP:");  
4     gets(buffer); // PERICOLOSO: non controlla la lunghezza!  
5 }
```

Le funzioni di libreria che *non controllano la dimensione* dei loro argomenti sono:

- `gets()` – legge da stdin senza limite di lunghezza.
- `strcpy()` – copia una stringa senza verificare la dimensione del buffer di destinazione.
- `sprintf()` – formatta una stringa in un buffer senza controllo di dimensione.

Contromisure al buffer overflow

- **Scrivere codice sicuro:** evitare le funzioni citate, usare le varianti sicure (`fgets`, `strncpy`, `snprintf`).
- **Non-Executable Stack (NX bit):** impedire l'esecuzione di codice nella zona stack. In Linux, il kernel con patch apposita previene l'esecuzione di codice arbitrario nello stack.
- **Controlli a tempo di compilazione:**
 - Patch ai compilatori che controllano la dimensione dei buffer (aumentano dimensione del codice).
 - **StackGuard** e **StackShield:** proteggono l'indirizzo di ritorno copiandolo in una posizione sicura e verificandolo prima del ritorno.
- **Controlli a run-time:** librerie come `libsafe` che verificano a runtime le dimensioni degli stack frame.

8.4.2 Errori Tempo Controllo / Tempo Utilizzo (TOCTOU)

Definizione – TOCTOU – Time of Check to Time of Use

Gli errori **TOCTOU** sfruttano la finestra temporale che intercorre tra il momento in cui un'entità *controlla* i permessi di accesso a una risorsa e il momento in cui quella risorsa viene *effettivamente usata*.

Scenario tipico:

1. **Fase 1:** L'entità *A* chiede all'entità *B* di eseguire l'operazione *op* sul file *a.txt*. *B* verifica le autorizzazioni di *A* su *a.txt* e le trova valide.

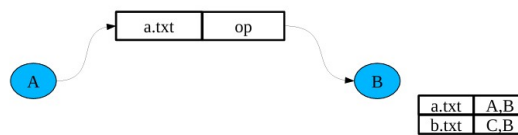


Figura 8.5

2. **Fase 2:** Nel lasso di tempo tra il controllo e l'uso, *A* sostituisce il riferimento (ad esempio tramite un link simbolico) in modo che punti al file *b.txt*. Quando *B* esegue l'operazione, essa viene applicata a *b.txt* invece che ad *a.txt*, violando la sicurezza.

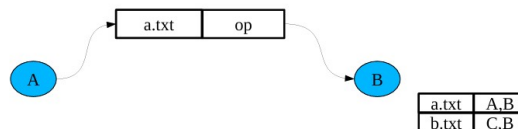


Figura 8.6

Esempio – TOCTOU in Unix – /tmp race condition

Un programma con privilegi elevati controlla i permessi su `/tmp/pippo.txt`, poi apre il file. Nel frattempo, un attaccante sostituisce `/tmp/pippo.txt` con un link simbolico a `/etc/passwd`. Il programma apre e modifica `/etc/passwd` credendo di lavorare su un file temporaneo.

8.5 Codice Maligno

8.5.1 Trojan Horse (Cavalli di Troia)

Definizione – Trojan Horse

Sono programmi che *replicano le funzionalità di programmi legittimi* o appaiono innocui, ma contengono codice maligno nascosto. Tipicamente catturano informazioni critiche (credenziali, dati sensibili) e le trasmettono al loro creatore, oppure compromettono o distruggono informazioni importanti.

Esempi di trojan horse:

- **Spyware**: raccolgono informazioni sul sistema e le inviano alla ditta produttrice (comune in software di file sharing).
- **Typosquatting via PATH**: file eseguibili con nomi simili a comandi di sistema (es. `dri` al posto di `dir`, `la` al posto di `ls`) posizionati in directory nel `$PATH`.
- **Login spoofing**: un programma che simula la schermata di login per rubare le credenziali (descritto in dettaglio nella sezione sull'autenticazione).

Nota – Easter Egg vs Codice Maligno

Gli *Easter egg* condividono il meccanismo di base di alcune minacce (come le bombe logiche o le backdoor, inserite dai creatori originali del software), ma **non hanno scopi malevoli**. Si tratta di funzionalità nascoste, messaggi, o minigiochi inseriti intenzionalmente dagli sviluppatori all'interno di un programma legittimo. Di solito si attivano tramite sequenze di comandi non documentate e vengono utilizzati spesso come "firma" o per includere i crediti (menzioni) del team di sviluppo.

8.5.2 Bombe Logiche

Definizione – Bomba Logica

Il creatore di un software inserisce del codice nascosto che si attiva al verificarsi di *condizioni specifiche*. La bomba logica rimane dormiente fino a quando la condizione di innesco non si verifica.

Esempio – Caso tipico di bomba logica

Uno sviluppatore inserisce nel software aziendale una bomba logica che si attiva quando il suo nome scompare dal database dei salari (cioè quando viene licenziato). Poiché non può più accedere al sistema, non può disattivarla. La bomba si attiva automaticamente causando danni.

Contromisure:

- analisi del codice sorgente (*code reviews*).
- Tuttavia, l'articolo "Reflections on Trusting Trust" di Ken Thompson dimostra che fidarsi completamente del codice sorgente è un problema più profondo di quanto sembri: anche un compilatore compromesso può inserire backdoor nel codice compilato, senza che queste siano visibili nel sorgente.

8.5.3 Backdoor e Trapdoor

Definizione – Backdoor / Trapdoor

Il creatore di un software può deliberatamente lasciare "porte di servizio" per entrare nel sistema aggirando i normali meccanismi di protezione.

Esempi storici:

- Il film *War Games* (1983) è basato su questo concetto.
- **Back Orifice, Sobig, MyDoom**: backdoor famose.
- Novembre 2003: tentativo (sventato) di inserire una backdoor nel kernel Linux.

La domanda aperta rimane: *cosa succede nei sistemi operativi proprietari?*

Contromisure: analisi del codice sorgente; nei sistemi open source la comunità può rilevare tentativi come quello del 2003 sul kernel Linux.

8.5.4 Virus e Worm

Definizione – Virus e Worm

- **Virus**: frammento di programma che *infetta altri programmi non maligni* modificandoli. Opera su qualunque supporto. Infettano programmi esistenti
- **Worm** (batterio): programma autonomo che diffonde *copie di sé stesso* in una rete. È un programma completo (autonomo), non parassita.

Le differenze tradizionali tra virus e worm sono oggi quasi completamente sfumate; la distinzione è più storica che pratica.

Ciclo di vita di un virus

1. **Riproduzione/Infezione**: il virus si diffonde il più possibile, cercando di bilanciare la velocità di infezione con il rischio di essere scoperto.
2. **Attivazione**: al verificarsi di un evento specifico, il virus esegue le proprie azioni dannose (o consuma risorse).

Come si diffonde un virus

Un virus deve essere *eseguito* per potersi diffondere. I vettori di diffusione sono:

- **Accodamento a programmi eseguibili:** file EXE/COM (MS-DOS), PE (Windows), ELF (Linux).
- **Boot sector e Master Boot Record (MBR):** il virus si esegue durante il boot del sistema.
- **Macro virus:** si diffonde tramite file Office o posta elettronica che supportano l'esecuzione automatica di macro.

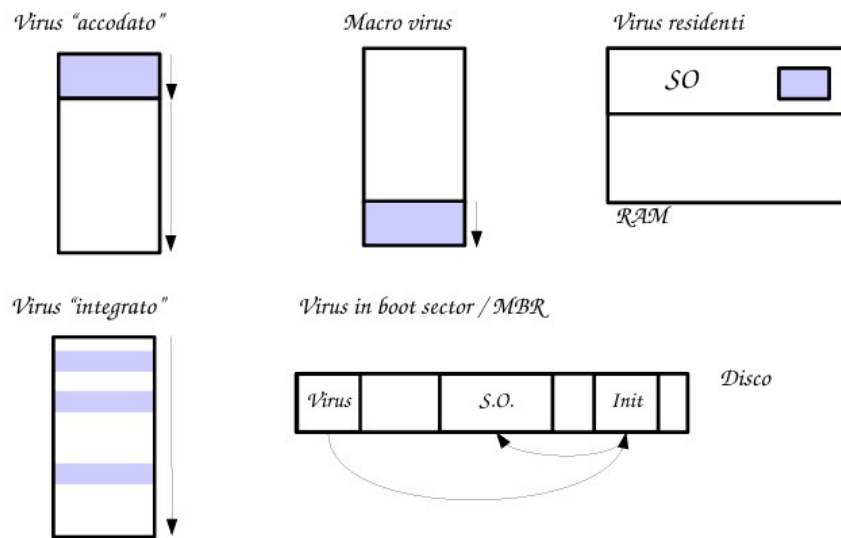


Figura 8.7: Tipologie di virus: *accodato* (aggiunge codice alla fine dell'eseguibile), *integrato* (inserisce codice all'interno), *macro virus* (nei documenti), *residente in RAM* (intercetta le chiamate al SO), *in boot sector/MBR* (si esegue al boot prima del sistema operativo).

Individuazione e occultamento

I virus non sono invisibili: per potersi diffondere e agire devono necessariamente essere memorizzati sul sistema ed eseguire azioni "tipiche" . Analizzando queste azioni, è possibile estrapolare una **firma** (*signature*) che caratterizza univocamente il virus; questa firma è la sequenza di byte che i classici programmi anti-virus cercano all'interno dei file .

Per sfuggire a questi controlli, i creatori di malware utilizzano tecniche di occultamento avanzate :

- **Virus stealth (invisibili):** ingannano attivamente il sistema operativo nascondendo le alterazioni apportate ai file. Lo fanno intercettando le *system call* di lettura o di interrogazione del file system (come `stat` o `read`). Se l'antivirus tenta di scansionare un file infetto, il virus "maschera" l'infezione fornendogli al volo la versione originale e pulita del file o nascondendo il suo aumento di dimensione.
- **Virus polimorfici:** sono progettati per mutare il proprio aspetto a ogni nuova infezione, eludendo il riconoscimento basato sulle firme statiche . Generalmente, il corpo del virus viene cifrato con una chiave sempre diversa. La piccola porzione di codice in chiaro necessaria per la decifratura (lo *stub*) viene mutata ogni volta: vengono scambiate istruzioni indipendenti, si utilizzano registri diversi della CPU o si aggiunge codice inutile (*junk code*). Di conseguenza, il file infetto compie le stesse azioni, ma il suo codice appare sempre completamente diverso all'analisi statica.

Alcune statistiche: esistono oltre 100.000 virus conosciuti, ma solo circa 1.000 sono "attivi" in un dato periodo.

8.5.5 Internet Worm di Morris (1988)



Figura 8.8: Meccanismo di propagazione del worm di Morris. Il worm nel sistema infetto sfrutta vulnerabilità in `rsh`, `finger` e `sendmail` per installare un `hook` nel sistema bersaglio, stabilisce una connessione e scarica una copia di sé stesso.

Definizione – Internet Worm di Morris – 1988

Rappresenta il primo caso documentato di *worm* su larga scala nella storia di Internet. Creato da Robert Morris, uno studente della Cornell University, non era progettato per essere distruttivo (non cancellava file), ma un errore logico nel meccanismo di replicazione causò un sovraccarico tale da rendere inutilizzabili circa 6.000 macchine (il 10% della rete dell'epoca) per circa 24 ore. Questo evento segnò il passaggio dalla fase "romantica" e fiduciosa della rete a quella della sicurezza moderna, portando alla creazione del **CERT**.

Analisi tecnica dei vettori di attacco

Il successo del worm fu dovuto alla capacità di sfruttare simultaneamente tre diversi difetti di sicurezza comuni nei sistemi UNIX dell'epoca:

- **Buffer Overflow in fingerd:** Il demone `fingerd` (utilizzato per ottenere informazioni sugli utenti remoti) utilizzava la funzione C `gets()` per leggere il nome dell'utente. Questa funzione non controlla la dimensione dell'input; il worm inviava una stringa di oltre 512 byte per saturare il buffer nello stack, sovrascrivere l'indirizzo di ritorno e forzare l'esecuzione di una *shell* remota.
- **Trapdoor in sendmail:** Il software di gestione della posta elettronica era configurato con una funzionalità di *debug* attiva di default. Inviando il comando `DEBUG` via SMTP, il worm poteva forzare il demone a trattare il corpo dell'e-mail non come testo, ma come un comando shell da eseguire con i privilegi del sistema.
- **Debolezza di rsh e rexec:** Il worm tentava di propagarsi sfruttando le relazioni di fiducia tra host (file `.rhosts` e `hosts.equiv`). Se un utente aveva configurato l'accesso senza password tra due macchine, il worm entrava automaticamente. In alternativa, utilizzava un piccolo dizionario interno di circa 400 parole per tentare un attacco di *password guessing* (forza bruta).

Attenzione – Il bug di replicazione

Il worm chiedeva a ogni macchina se fosse già infetta. Per evitare che un amministratore fingesse un'infezione per proteggersi, Morris programmò il worm affinché si replicasse comunque una volta su sette. Questo errore di calcolo portò alla generazione di migliaia di processi sullo stesso sistema, esaurendo le risorse della CPU e della memoria (Denial of Service).

8.6 Autenticazione

Definizione – Autenticazione

L'**autenticazione** è l'operazione di associare ad un utente una *identità verificata*. È il problema alla base di tutti i meccanismi di protezione: senza autenticazione, autorizzazione e controllo degli accessi sarebbero inutili.

I meccanismi di autenticazione si basano su tre categorie di fattori:

1. Qualcosa che l'utente **conosce** (password, PIN).
2. Qualcosa che l'utente **ha** (oggetto)(tessera bancomat, smart card, token OTP).
3. Qualcosa che l'utente **è** (peculiarità di una persona)(biometria: impronta digitale, retina, voce).

8.6.1 Autenticazione tramite qualcosa che l'utente conosce: password

L'autenticazione basata su password è la più diffusa. I principali problemi sono:

- **Password indovinate**: cultura della sicurezza carente, password banali, post-it visibili.
- **Password carpite**: shoulder surfing, login spoofing, sniffing di rete.

Memorizzazione delle password

- **In chiaro** (sistemi obsoleti): le password sono memorizzate in un file leggibile; troppo pericoloso.
- **Codificate con OWF (one way function)**: il file delle password contiene $f(\text{password})$. Al login, si calcola $f(\text{input})$ e lo si confronta con il valore memorizzato. Il file in Unix/Linux è `/etc/passwd`, (oggi: `/etc/shadow`).

Attacco al dizionario (cracking)

Dizionario		Password file
in chiaro	codificate	
280771	al47r33km	<pre>rossi:8kfdjkj kf:500:500:... franchi:rlekek1139:501:501:...</pre>
password	ldcfjoiu3	
rossi	rlekek1139	
franchi	8kfdjkj kf	
gelato	39u4jrk0	

Figura 8.9: Attacco al dizionario: le password del dizionario vengono cifrate con la stessa OWF e confrontate con i valori nel password file. Se una password nel file coincide (es. **franchi** → **rlekek1139**), l'attacco ha successo.

Il meccanismo di Salt

Per rendere più costoso l'attacco al dizionario si utilizza il **salt**:

Definizione – Salt

Prima di essere cifrata e memorizzata, la hash della password viene *concatenata con un numero casuale* (salt). Il salt viene memorizzato in chiaro nel file delle password. In questo modo, l'attaccante deve ri-calcolare l'intero dizionario per ogni possibile valore di salt, moltiplicando enormemente il costo dell'attacco.

Esempio – Formato con salt in /etc/passwd

```
pippo:8kfdjkjhf/2000:500:500:Pippo Baudo:/home/pippo:/bin/bash
```

Il campo dopo i due punti contiene la password cifrata; il salt è 2000.

Shadow Password File

Definizione – Shadow Password

Il file /etc/passwd deve essere leggibile da tutti (contiene informazioni come la home directory e la shell). Per evitare che le password cifrate siano accessibili a tutti, il *meccanismo shadow* memorizza le password in un file separato (/etc/shadow) leggibile *solo da root*.

Esempio – /etc/passwd con shadow

```
tizio:x:501:502:Vittorio Tizio:/home/tizio:/bin/bash
caio:x:502:503:Aristide Caio:/home/caio:/bin/bash
```

La x nel secondo campo indica che la password è in /etc/shadow.

Scelta di password sicure – Test di Morris-Thompson

Una password è debole se:

- È più corta di 7 caratteri.
- Contiene solo caratteri alfabetici (tutti minuscoli o tutti maiuscoli).
- È una parola di dizionario (anche con sostituzioni banali: 1 per i, 0 per o, 3 per e; o scritta al contrario).
- È una data di nascita.

Regole per una password robusta: almeno 8 caratteri, con lettere minuscole, maiuscole, cifre numeriche e caratteri speciali.

Attenzione – Statistiche allarmanti

- Morris-Thompson (1979): l'86% delle password testate risultava identificabile.
- Klein (1990): 25% delle password in sistemi Unix UK/USA.
- Kabay (1997): 82% delle password nel financial district di Londra.

8.6.2 Attacchi alle password

Login Spoofing

1. L'attaccante avvia un programma che mostra una schermata di login fasulla (identica a quella reale).
2. Attende che la vittima inserisca le credenziali.
3. Le memorizza (o le invia all'attaccante remoto).
4. Visualizza "Login incorrect" e fa partire il vero programma di login.
5. La vittima crede di aver digitato male la password, reinserisce le credenziali nel login legittimo ed entra normalmente, senza accorgersi di nulla.

Esempio – Script di login spoofing in csh

```
1  #!/bin/csh -f
2  # Solo per dimostrazione
3  cat /etc/issue
4  echo -n "hostname_login:_"
5  set x="$<"
6  stty -echo
7  echo -n "Password:_"
8  set y="$<"
9  echo $x "," $y >> captured.dat
10 stty -echo
11 echo ""
12 echo Login incorrect
13 echo ""
14 exit
```

Il file `captured.dat` contiene le credenziali rubate.

Packet Sniffing

Un **packet sniffer** analizza il traffico di rete e cerca di individuare pacchetti contenenti coppie login/password trasmesse in chiaro (come avviene con `telnet` e `ftp` tradizionali).

Challenge-Response

Definizione – Autenticazione Challenge-Response

Schema basato su funzione one-way f con chiave k nota ad entrambe le parti:

1. Il sistema genera una *challenge* c (numero casuale) e la invia all'utente (quando quest'ultimo richiede accesso).
2. Sia il sistema che l'utente calcolano $f(c, k)$.
3. L'utente invia al sistema il valore $f(c, k)$.
4. Il sistema confronta il valore ricevuto con quello calcolato localmente.

Poiché ad ogni sessione viene trasmessa solo $f(c, k)$ (mai la password k), e c è sempre diverso, il packet sniffing diventa inutile: il valore intercettato non può essere riutilizzato.

8.6.3 Password One-Shot

L'utente dispone di un elenco stampato di password, usa ciascuna una sola volta. Usato in contesti militari e da alcuni istituti bancari, è efficace solo se la lista fisica è protetta.

8.6.4 Autenticazione con oggetti fisici e biometrica

- **Tessera bancomat:** facile da copiare; la sicurezza si basa sul doppio fattore PIN + carta.
- **Smart card:** difficile da copiare, dispone di unità di calcolo propria; supporta meccanismi challenge-response.
- **Biometria:** impronta digitale, retina, voce. Vantaggi: non si dimentica e non si perde; svantaggi: non modificabile se compromessa, possibili falsi positivi/negativi.

8.6.5 PAM – Pluggable Authentication Modules

Definizione – PAM

PAM (Pluggable Authentication Modules) è un servizio generale di autenticazione per sistemi UNIX (demone usato da demoni), basato su file di configurazione e moduli liberamente caricabili. Permette di separare i meccanismi di autenticazione dai servizi che li utilizzano.

È standard in: Linux, FreeBSD, HP-UX, Solaris, macOS. AIX utilizza LAM (simile ma proprietario IBM).

Componenti di PAM

- **Servizi:** programmi PAM-aware (`login`, `passwd`, `su`, `sshd`, ...) che effettuano chiamate ai moduli tramite l'API PAM.
- **Moduli:** implementano i task di autenticazione, gestione account, gestione password e logging.
- **File di configurazione:** in `/etc/pam.d/` (uno per servizio) o `/etc/pam.conf`.

Struttura dei file di configurazione

Ogni riga ha la forma: `classe flag modulo [opzioni]`

Classi di operazione:

- **auth:** procedure per l'autenticazione dell'utente.
- **account:** modifica degli attributi di un account (dà limiti temporali all'account e vede se è attivo o no).
- **password:** modifica della password.
- **session:** debug e logging su syslog.

Flag di controllo:

- **sufficient:** se il modulo ritorna concesso(`grant`), la risposta è positiva e i moduli successivi vengono ignorati.
- **required:** se il modulo nega(`deny`), la risposta è immediatamente negativa.
- **required:** il modulo deve concedere affinché l'operazione abbia successo (ma l'esito non è immediato: si valutano anche i moduli successivi).
- **optional:** influisce solo se è l'unico modulo presente per quella classe.

Esempio – Esempio: `/etc/pam.d/su`

```
%PAM-1.0
auth    sufficient  pam_rootok.so    // se è già root non serve autenticazione
auth    required    pam_unix2.so     // nullok e continua a guardare i successivi
account required    pam_unix2.so
password required    pam_pwcheck.so    nullok
```

```
password required    pam_unix2.so      nullok use_first_pass use_authok
session required    pam_homecheck.so
session required    pam_unix2.so      debug
Se chi esegue su è già root (pam_rootok.so → sufficient), l'autenticazione ha
successo senza richiedere la password.
```

8.7 Controllo degli Accessi: Autorizzazione

8.7.1 Protezione del nucleo e autorizzazione

Il sistema operativo implementa due livelli di protezione:

- **Protezione del nucleo:** meccanismi hardware (mode bit kernel/user, interrupts, protezione memoria e dispositivi) che separano il gestore dalle risorse gestite.
- **Autorizzazione:** meccanismi e politiche con cui il SO decide se un soggetto ha il permesso di eseguire una determinata azione su un oggetto (implementati via Trusted Computing Base, Reference Monitor, Matrice di Accesso).

8.7.2 Principi fondamentali

Definizione – Principi del controllo degli accessi

- **Reference Monitor (Accesso Mediato):** *tutti* gli accessi ad un oggetto devono essere controllati. In pratica, molti OS controllano i diritti solo all'apertura del file, non per le operazioni successive.
- **Separazione dei privilegi:** il sistema non dovrebbe concedere permessi in base ad una singola condizione. Per esempio UNIX permette ad un utente di diventare root se e solo se conosce la password di root e fa parte del gruppo wheel.
- **Failsafe Default:** nessun soggetto ha diritti per default; i permessi vengono concessi esplicitamente.
- **Privilegio Minimo (Need-to-know):** ogni soggetto possiede, in ogni istante, solo i diritti necessari per la fase corrente dell'elaborazione.

8.7.3 Definizioni: soggetti, oggetti, diritti, dominio

Definizione – Entità del controllo degli accessi

- **Soggetti S :** entità *attive* che eseguono azioni (processi, thread).
- **Oggetti O :** entità (passive/attive) su cui vengono eseguite azioni (file, dispositivi, processi). In generale $S \subseteq O$.
- **Diritti di accesso R :** azioni che possono essere eseguite (**read**, **write**, **execute**).
- **Dominio di protezione:** insieme di coppie $\langle o, r_s \rangle$ con $o \in O$ e $r_s \subseteq R$; definisce cosa un soggetto può fare su un oggetto.
In UNIX Il dominio di ogni dominio è determinato da *user-id* e *group-id*.
In Java la sicurezza è basata su un dominio per *classe*

Attenzione

in generale associamo un dominio per processo

8.7.4 Matrice di Accesso

Definizione – Matrice di Accesso

La **matrice di accesso** è una matrice domini/oggetti dove l'elemento $A(i, j)$ contiene i diritti che il dominio D_i prevede per l'oggetto O_j . Quando viene creato un nuovo oggetto, si aggiunge una colonna alla matrice con i diritti stabiliti al momento della creazione.

oggetti domini	File1	File2	File3	Stampante
D1	read			
D2		read write	read	
D3			read write	write
D4				write

Figura 8.10: Esempio di matrice di accesso: i domini D_1 – D_4 rappresentano thread o processi, le colonne rappresentano oggetti (file, stampante). D_1 può solo leggere File1; D_3 può leggere e scrivere File3 e scrivere sulla stampante.

Associazione soggetto/dominio: l'associazione può essere statica o dinamica. Esempi di associazione dinamica:

- Modalità user/kernel nell'esecuzione dei processi.
- Bit SETUID di Unix.
- Modalità "Run as" di Windows.

Implementazione della Matrice di Accesso: ACL vs Capability

La matrice di accesso teorica è concettualmente semplice, ma nella realtà implementativa risulta enorme e fortemente sparsa. Per ottimizzare l'occupazione di memoria, i sistemi operativi moderni scompongono la matrice proiettandola per righe o per colonne.

- **ACL – Access Control List (Proiezione per Colonne):** Si scompone la matrice associando a ogni *oggetto* (colonna) la lista dei soggetti (domini) autorizzati e dei relativi diritti: $\langle \text{dominio, diritti di accesso} \rangle$. Metaforicamente, è come avere una **lista degli invitati** appesa alla porta di una stanza: la guardia (il sistema) controlla se chi vuole entrare è presente nell'elenco.
- **Capability (Proiezione per Righe):** Si scompone la matrice associando a ogni *dominio/soggetto* (riga) l'elenco delle risorse a cui può accedere. Una *capability* è una coppia $\langle \text{oggetto, diritti di accesso} \rangle$.

Nota

I processi mantengono le capability e le presentano quali credenziali per accedere agli oggetti

Metaforicamente, il soggetto possiede un **mazzo di chiavi**; ogni chiave apre una specifica porta. Se un processo presenta la chiave corretta come credenziale, vi accede.

	ACL (Access Control List)	Capability
Memorizzazione	Presso l'oggetto.	Presso il soggetto (o processo).
Vantaggio principale	Facile rispondere alla domanda: <i>"Chi può accedere a questo file?"</i>	Facile rispondere alla domanda: <i>"A cosa può accedere questo processo?"</i>
Efficienza di accesso	Più lenta: il sistema deve scorrere la lista a ogni richiesta.	Molto veloce: basta presentare la chiave corretta.
Revoca dei diritti	Semplicissima: basta aggiornare le strutture dati cancellando l'utente dalla lista dell'oggetto.	Molto complessa: l'informazione risiede presso i processi, bisogna rintracciare e annullare chiavi già distribuite.

8.7.5 Revoca dei diritti di accesso

La revoca dei diritti di accesso concessi a un soggetto può essere classificata secondo diverse dimensioni:

- **Immediata** o **ritardata** (subito o si può attendere).
- **Selettiva** (per alcuni domini) o **generale** (per tutti).
- **Parziale** (solo alcuni diritti) o **totale** (tutti i diritti).
- **Temporanea** o **permanente**.

Revoca nei sistemi basati su ACL: Risulta un'operazione banale, in quanto è sufficiente aggiornare in modo corrispondente le strutture dati dei diritti di accesso cancellando o modificando i permessi dell'utente presso l'oggetto.

Nota – Il Problema della Revoca nei Sistemi a Capability

Nei sistemi basati su capability, l'informazione relativa ai permessi è distribuita e memorizzata presso i processi. Questo rende la revoca intrinsecamente complessa. Per affrontarla si utilizzano diverse strategie:

- **Capability a validità temporale limitata:** Le chiavi "scadono" automaticamente dopo un prefissato periodo di tempo, realizzando di fatto una revoca ritardata.
- **Doppia memorizzazione:** Ogni capability viene controllata dal sistema prima di essere utilizzata. Questo approccio garantisce la revoca sicura, ma fa perdere i benefici di efficienza tipici del modello a capability.
- **Capability indirette:** Vengono concessi diritti non direttamente agli oggetti, ma a elementi di una tabella globale intermedia gestita dal sistema. Cancellando l'elemento dalla tabella, si revocano istantaneamente i permessi associati.
- **Cambiamento dell'identità dell'oggetto:** Si utilizza un contatore nel nome dell'oggetto. Alla revoca, il contatore cambia e le vecchie chiavi diventano invalide, costringendo i processi a richiedere una nuova autorizzazione (risulta oneroso in caso di variazioni frequenti).

Integrità delle Capability Poiché le capability funzionano come chiavi fisiche, il meccanismo regge solo se non possono essere "coniate" o falsificate in alcun modo dai processi. Esistono due approcci principali per garantirne la sicurezza:

- **Protezione nello Spazio Kernel:** Le capability sono mantenute e protette dai meccanismi hardware del kernel (es. Hydra, o parzialmente le POSIX capability in Linux).
- **Protezione in Spazio Utente:** Le capability sono memorizzate dai processi nello spazio utente, ma sono protette da meccanismi crittografici che ne impediscono la modifica non autorizzata (tipico dei sistemi distribuiti).

8.7.6 Modello tradizionale UNIX (modello ACL limitato)

Identificatori di processo

Ogni processo UNIX possiede almeno sei ID:

Definizione – ID di un processo UNIX

- **Real User ID / Real Group ID**: identificano il vero utente e gruppo che esegue il processo; non cambiano durante la vita del processo.
- **Effective User ID / Effective Group ID / Supplementary GIDs**: determinano effettivamente i diritti di accesso al filesystem.
- **Saved Set-User-ID / Saved Set-Group-ID**: copie dell'effective UID/GID, inizializzate dalla syscall `setuid`; permettono di passare temporaneamente a un diverso insieme di privilegi.

Bit SETUID e SETGID

Normalmente `effective_uid = real_uid`. In alcuni casi è necessario che un processo abbia più diritti di chi lo esegue:

Esempio – Comando `passwd`

Il comando `passwd` deve essere eseguibile da qualunque utente ma deve poter modificare `/etc/passwd` (di proprietà di root). Il bit SETUID permette questo:

- Owner del file `/usr/bin/passwd`: root.
- Quando eseguito, `effective_uid = 0` (root).
- Il processo può scrivere su `/etc/passwd`.

Definizione – Bit SETUID e SETGID

- **Set-User-ID (SUID)**: su un file eseguibile, forza `effective_uid = owner_del_file`.
- **Set-Group-ID (SGID)**: su un file eseguibile, forza `effective_gid = owning_group_del_file`. Su una directory, i nuovi file creati ereditano il group owner della directory.

Algoritmo di verifica degli accessi

Algoritmo UNIX per la verifica dei permessi di accesso a un file. Root bypassa tutti i controlli; per gli altri, si verifica in sequenza: owner, group, others.

L'algoritmo di accesso in pseudocodice:

1. IF `effective_uid == 0` (root) $\Rightarrow$ **accesso concesso**.
2. ELSE IF `effective_uid == owner` AND permesso utente appropriato $\Rightarrow$ **accesso concesso**.
3. ELSE IF `group_owner  $\in$  supplementary_gids  $\cup$  {effective_gid}` AND permesso gruppo appropriato $\Rightarrow$ **accesso concesso**.
4. ELSE IF permesso "others" appropriato $\Rightarrow$ **accesso concesso**.
5. ELSE $\Rightarrow$ **accesso negato**.

Sticky Bit e umask

Definizione – Sticky Bit

Il **sticky bit** che è il 12° bit (detto anche *saved text mode*, indicato con `t` in `ls -l`) applicato a una directory impedisce agli utenti di cancellare file che non appartengono loro, anche se hanno i diritti di scrittura sulla directory. Esempio classico: `/tmp`.

Definizione – umask

La **umask** è una maschera applicata automaticamente ogni volta che un processo crea un nuovo file. Tutti i bit attivi nella maschera vengono *spenti* nei permessi del file creato.

Esempio – Effetto della umask

Creazione di un file con diritti `rw-rw-rw-`:

Mode	R	W		R	W		R	W		
Mask										
Final	R	W		R	W		R	W		

umask
000

Mode	R	W		R	W		R	W		
Mask					W		R	W		
Final	R	W		R						

umask
046

Figura 8.11

- Con umask 000: permessi finali `rw-rw-rw-` (nessun bit rimosso).
- Con umask 046: permessi finali `rw-r---` (rimossi i permessi `group:w` e `others:rw`).

Limiti del modello UNIX tradizionale

Il modello UNIX tradizionale è semplice ed efficiente (12 bit per file), ma presenta limiti:

- Non permette ACL per singoli utenti su file già assegnati a un gruppo.
- Il meccanismo SETUID può portare facilmente a sistemi compromessi.
- Viola il principio del privilegio minimo.
- Alcune applicazioni (FTP, Samba) devono implementare le proprie estensioni.

8.7.7 POSIX ACL

Definizione – POSIX ACL

Le **POSIX ACL** estendono il modello di permessi UNIX tradizionale, permettendo di specificare diritti di accesso per singoli utenti e gruppi specifici su ogni file o directory. Sono disponibili nativamente a partire dai kernel 2.5–2.6+ e supportate dai principali filesystem come Ext2/3/4, JFS, ReiserFS e XFS.

Tipi di ACL entry

Nelle ACL, i permessi di ogni oggetto sono rappresentati da un insieme di entry:

Tipo	Formato
Owner	<code>user::rwx</code>
Named user	<code>user:nome:rwx</code>
Owning group	<code>group::rwx</code>
Named group	<code>group:nome:rwx</code>
Mask	<code>mask::rwx</code>
Others	<code>other::rwx</code>

ACL minime ed estese

Il sistema distingue due categorie principali di ACL:

- **ACL minima (di UNIX)**: contiene solamente tre entry (`user::`, `group::`, `other::`). Il suo comportamento è esattamente equivalente a quello dei bit di permesso del modello UNIX tradizionale.
- **ACL estesa**: possiede più di tre entry e contiene obbligatoriamente una entry `mask`. Può contenere un numero arbitrario di *named user* e *named group*.

Nelle ACL estese, tutti i *named user* e *named group* rientrano logicamente nella classe *group*. In questo contesto, i permessi definiti nella `mask` giocano un ruolo fondamentale: rappresentano il limite superiore (*upper bound*) dei permessi che possono essere effettivamente concessi a qualsiasi elemento appartenente alla classe gruppo.

Visualizzazione delle ACL minime

▪ ACL minime

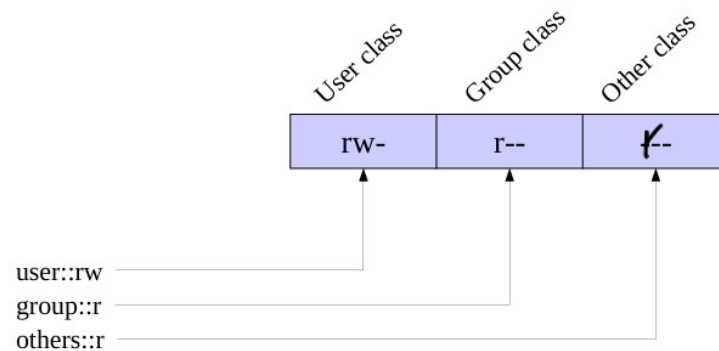


Figura 8.12: Rappresentazione di un'ACL minima corrispondente ai permessi `rw-r---`

Nelle **ACL minime**, la struttura dei permessi rimane identica a quella classica di UNIX, mappando i bit di protezione direttamente sulle tre classi di utenti fondamentali:

- **User class:** Corrisponde alla entry `user::rw-`, definendo i diritti del proprietario del file
- **Group class:** Corrisponde alla entry `group::r-`, definendo i diritti per il gruppo proprietario
- **Other class:** Corrisponde alla entry `others::--`, definendo i diritti per tutti gli altri utenti

Visualizzazione delle ACL estese e della Maschera

▪ ACL estese

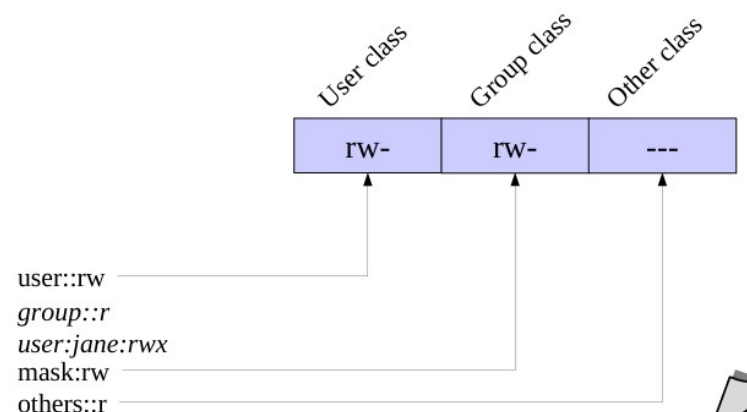


Figura 8.13: Mappatura di un'ACL estesa con utente nominativo e maschera.

Quando viene introdotta una **ACL estesa**, la gestione della *Group class* cambia radicalmente per accogliere i nuovi privilegi:

- **Inclusione nella Group class:** Oltre all'owning group (`group::r-`), anche i *named user* (come `user:jane:rwX`) vengono raggruppati logicamente sotto i permessi della classe gruppo
- **Il ruolo della Mask:** Viene introdotta la entry `mask::rw-`, che funge da filtro per tutti i membri della classe gruppo
- **Bit di permesso di ls:** In presenza di ACL estese, i bit della classe gruppo visualizzati dal sistema non mostrano più i permessi dell'owning group, bensì il valore della **maschera**

Esempio – ACL estesa – effetto della mask

```
user::rw-  
group::r--  
user:jane:rwX  
mask::rw-  
other::r--
```

L'utente *jane* possiede i diritti `rwX` nella sua specifica entry, ma la *mask* per la classe gruppo è impostata a `rw-`. Il permesso effettivo di *jane* sarà quindi `rw-` (calcolato tramite l'intersezione tra i permessi richiesti e la *mask*, se viene chiesto di più quindi gli viene negato). L'owning group, allo stesso modo, avrà un diritto effettivo pari a `r-`

Default ACL ed ereditarietà

Per gestire l'ereditarietà dei permessi nelle gerarchie di cartelle, si utilizzano due concetti:

- **Access ACL:** è associata a ogni singolo file (regolare e non) e ne determina attivamente i diritti di accesso .
- **Default ACL:** è associata esclusivamente alle directory e determina i diritti predefiniti dei file creati al loro interno

Quando un nuovo file o una nuova sottodirectory vengono creati, la *default ACL* della cartella padre viene copiata nella *access ACL* del nuovo elemento (e anche nella sua *default ACL*, se è una directory). Se vengono specificati dei diritti al momento della creazione, il sistema ne effettua l'intersezione con la default ACL

Algoritmo di access control POSIX

Quando un processo p tenta di accedere a un file f richiedendo un tipo di accesso t , il sistema operativo applica questo algoritmo in due passi:

1. **Selezione:** il sistema cerca la singola ACL entry di f che identifica p nel modo più specifico.

La ricerca avviene rigorosamente in quest'ordine: *owner, named users, owning or named groups, others* . Non appena trova una corrispondenza (ad esempio analizzando i gruppi uno a uno finché non ne trova uno che contiene t), si ferma e seleziona quell'entry.

2. **Verifica:** il sistema controlla se l'entry selezionata al passo precedente concede effettivamente il tipo di accesso t . Se l'entry fa parte della classe gruppo, il permesso viene prima intersecato con la *mask*

Implementazione nei filesystem

Tecnicamente, le ACL non sono uno standard POSIX base, ma sono implementate tramite gli **Extended Attribute (EA)**, ovvero coppie *nome=valore* associate agli inode dei file . Le implementazioni variano in base al filesystem:

- **Ext2/3/4:** gli EA sono memorizzati in un blocco separato puntato dall'inode (con possibilità di sharing tra blocchi uguali). Hanno un limite di 1 KB, permettendo un massimo di 25–32 entry.
- **JFS:** gli EA risiedono in un extent puntato dall'inode (o nell'inode stesso se sono piccoli). Non hanno un limite di dimensione intrinseco, permettendo fino a 8191 entry.
- **XFS:** utilizza l'inode per gli EA piccoli e una complessa ma efficiente struttura B-tree per quelli più grandi. Max 25–32 entry .
- **ReiserFS:** implementa gli EA come piccoli file (il cui nome è associato all'inode) salvati all'interno di una directory di sistema nascosta . Max 8191 entry .

Nota – Considerazioni pratiche sulle prestazioni

Utilizzare ACL molto grandi (con moltissime entry) riduce inevitabilmente le prestazioni del filesystem. Nella pratica, è sempre preferibile organizzare accuratamente gli utenti in gruppi UNIX standard, ricorrendo alle ACL estese solo per gestire eccezioni granulari. Inoltre, mentre i tool standard (*cp*, *ls*, *tar*) supportano nativamente le ACL, alcune vecchie applicazioni o comandi (come *cpio* o *scp*) potrebbero non gestirle correttamente, perdendo i metadati durante i trasferimenti.

La compatibilità tra NTFS ACL (Windows) e POSIX ACL non è totale, ma le versioni recenti di SAMBA offrono una buona interoperabilità .

8.7.8 POSIX Capability

Definizione – POSIX Capability

Le **POSIX Capability** “spezzano” il ruolo onnicomprensivo del superutente in un insieme di *sotto-privilegi granulari*. Un processo eseguito come root ottiene solo le capability strettamente necessarie, non tutti i privilegi del superutente.

Alcune delle capability definite in Linux:

Capability	Descrizione
CAP_CHOWN	Modificare permessi dei file
CAP_DAC_OVERRIDE	Bypassare tutte le restrizioni DAC(ovvero consente accesso)
CAP_DAC_READ_SEARCH	Bypassare le restrizioni di lettura/ricerca DAC
CAP_KILL	Inviare segnali a processi altrui
CAP_SETUID	Modificare l'UID del processo
CAP_NET_BIND_SERVICE	Aprire porte privilegiate (< 1024)

Ogni processo possiede tre bitmap di capability:

- **Effective Set:** le capability attualmente attive. Contenuto nel Permitted Set.
- **Permitted Set:** il massimo insieme di capability possedibili.
- **Inheritable Set:** le capability trasmissibili ai sottoprocessi.

L'uso di capability sta prendendo piede in programmi come `sendmail` e `bind`. Hanno ricevuto trattamento prioritario con l'introduzione dei **Linux Security Modules (LSM)** nel kernel 2.6.

8.7.9 DAC e MAC

Definizione – DAC – Discretionary Access Control

Il **DAC** basa i controlli di accesso sull'*identità dei soggetti* e sui permessi assegnati agli oggetti. Il *proprietario* di un oggetto ha diritti OWNER/COPY su di esso e può concedere l'accesso ad altri a sua discrezione.

Definizione – MAC – Mandatory Access Control

Il **MAC** basa i controlli su *regole* e informazioni associate a soggetti e oggetti (dette *livelli di sicurezza* o *etichette*). Il proprietario non ha privilegi speciali: le etichette sono assegnate esternamente e non sono controllate dal proprietario.

Quando DAC è sufficiente:

- Il sistema non contiene dati che debbano essere protetti da eventuali abusi del proprietario stesso
- Tutte le informazioni possono essere condivise liberamente a totale discrezione del proprietario dei dati

Quando DAC è insufficiente:

Esempio – Problema del confinamento dell'informazione in DAC

Alice possiede il file *A* e autorizza Bob alla lettura .

Bob legge *A* e lo copia nel file *B* .

Bob può quindi autorizzare Carl a leggere *B* .

In questo modo, Alice perde il controllo sulla propagazione delle informazioni originali contenute in *A*: il modello DAC non è in grado di prevenire questo scenario.

8.7.10 MAC – Modello Bell-LaPadula

Definizione – Modello Bell-LaPadula (BLP)

Il modello **Bell-LaPadula** è un sistema di controllo d'accesso obbligatorio (MAC) orientato esclusivamente alla **confidenzialità** (riservatezza) delle informazioni. Sviluppato originariamente per l'ambito militare, si basa su una gerarchia rigida di livelli di sensitività.

I livelli di sicurezza seguono un ordine lineare:

$$\text{Unclassified (U)} < \text{Confidential (C)} < \text{Secret (S)} < \text{Top Secret (TS)}$$

Il sistema assegna un'etichetta (livello di sicurezza) a ogni entità:

- **Soggetti (S):** Il livello $L(S)$ rappresenta la *security clearance*. Si distingue tra **livello massimo** (la clearance massima autorizzata) e **livello corrente** (il livello a cui il soggetto sta operando in un dato momento, che può essere inferiore al massimo).
- **Oggetti (O):** Ogni oggetto possiede un **unico livello di sicurezza** $L(O)$ che rispecchia i suoi requisiti di protezione.

Definizione – Proprietà fondamentali di BLP

Il modello impone due regole principali per prevenire la fuga di informazioni classificate:

- **Simple Security Property (“No reads up”):** Un soggetto S può leggere un oggetto O se e solo se $L(O) \leq L(S)$ e S ha il permesso di leggere O . Questa regola impedisce l'accesso a informazioni con classificazione superiore alla propria clearance.
- ***-Property (“No writes down”):** Un soggetto S può scrivere un oggetto O se e solo se $L(S) \leq L(O)$ e S ha il permesso di scrivere O . Questa regola impedisce a un soggetto di “declassare” informazioni sensibili scrivendole in oggetti con livello inferiore.

Il principio sintetico del modello è: *“Information flows up, not down”* (l'informazione può solo salire di livello, mai scendere).

Il flusso delle informazioni e l'intuizione del modello

L'interazione tra le due proprietà garantisce che la riservatezza sia mantenuta anche in presenza di codice maligno. In sintesi:

- Il **Read Down** è permesso (un generale può leggere un documento non classificato).
- Il **Write Up** è permesso (un soldato può scrivere informazioni in un rapporto destinato ai superiori).
- Il **Read Up** e il **Write Down** sono categoricamente vietati.

Nota – Perché è necessaria la *-Property?

Mentre la Simple Security Property è intuitiva, la ***-Property** serve a prevenire attacchi tramite **Trojan Horse**. Senza di essa, un programma maligno eseguito da un utente con alta clearance (es. Secret) potrebbe leggere dati segreti e scriverli in un file pubblico (Unclassified), permettendo a un utente non autorizzato di leggerli. Impedendo la scrittura verso il basso, il modello BLP confina l'informazione al suo livello di sicurezza o superiore.